

Etapa 2 — Massa de Dados de Treinamento

Autor

Marcus Vinícius Santos de Almeida

Data de entrega

11/09/2026

Objetivo da etapa

Gerar sinteticamente 100+ registros de treinamento, com padrões intencionais e distribuição de classes razoável — N = 1.000.000 nesta versão

Artefatos

`dados/gerar_dados.py` `dados/etapa2_dados_treinamento.csv`
`dados/etapa2_validacao.txt`

1 Metodologia de geração

1.1 Por que dados sintéticos

A Etapa 2 do enunciado exige explicitamente que os 100+ registros sejam **gerados por IA / sinteticamente**, e não coletados de repositórios reais. A geração é feita por um **script Python** (`dados/gerar_dados.py`) para que o modelo generativo seja deliberado, inspecionável e reproduzível.

1.2 Decisão de projeto: "Naive Bayes puro" — features independentes entre si

A decisão tomada para este projeto (documentada no `CLAUDE.md`) foi **gerar as 6 features de forma estatisticamente independente entre si**: nenhuma feature lê o valor de outra no gerador — sem tendência cruzada, sem ruído compartilhado, sem subpopulação de interação plantada. Só o **rótulo** (`defeito` = SIM/NÃO) depende das 6 features, o que é esperado e correto — é literalmente o que o Naive Bayes modela, $P(\text{feature} \mid \text{classe})$.

Por que essa escolha, e não plantar as correlações que a literatura documenta:

- Testa o classificador no cenário em que sua própria suposição central (independência condicional) é **verdadeira por construção** dos dados — o "Naive Bayes em sua forma pura".
- É uma escolha de simulação **legítima e declarada como tal** — não uma afirmação sobre como o mundo real funciona. A literatura (Shepperd, 1988; $R^2 \approx 0,93$ entre complexidade e LOC) documenta que, em código real, essas features **seriam** correlacionadas. Este dataset opta por não reproduzir essa correlação, de propósito, para isolar o comportamento teórico do algoritmo — discutido na Reflexão Crítica da Etapa 4.

Mecanismo: cada uma das 6 features é sorteada por um gerador de números aleatórios **próprio** (`numpy.random.Generator.spawn()`), uma "ramificação" independente da mesma semente `SEED`), evitando o compartilhamento acidental de ruído entre colunas.

1.3 Parâmetros principais

Parâmetro	Valor	Significado
N_REGISTROS	1.000.000	Nº de módulos gerados (mínimo exigido: 100). Aumentado da versão anterior (150) — ver CLAUDE.md, "Migração para DuckDB e N = 1.000.000 registros"
SEED	42	Semente única — todo gerador aleatório deriva dela (<code>rng.spawn</code>), garantindo reprodutibilidade
PROPORCAO_DEFEITO	0,35	Fração de módulos com <code>defeito = SIM</code> (longe de 0,5 e dos extremos)

Nota sobre o CSV gerado

Com N = 1.000.000, `dados/etapa2_dados_treinamento.csv` passa a ter dezenas de MB e **não é versionado no Git** — está listado no `.gitignore`. O arquivo é 100% reprodutível: rodando `python3 dados/gerar_dados.py` localmente, com a mesma semente (`SEED = 42`), qualquer pessoa obtém exatamente o mesmo CSV byte a byte. A avaliação desta atividade é sobre a apresentação oral, não auditoria do repositório Git — essa decisão não representa risco de entregável ausente.

1.4 Modelo generativo (resumo)

LOC vem de uma log-normal (muitos arquivos pequenos, poucos grandes). **Complexidade** é uma log-normal própria (mediana $\approx 8,6$), com uma subpopulação aleatória (12%) de complexidade desproporcionalmente alta — sorteio independente, que não olha LOC nem nenhuma outra coluna. **Nº de autores** é uma contagem de Poisson de média fixa. **Churn relativo** é log-normal em torno de $\sim 10,5\%$. **Nº de imports** soma um nível médio fixo $\times$ ruído log-normal forte + uma contagem de Poisson independente. **Cobertura** é normal em torno de 63%. O **rótulo** é a soma de pesos (log-odds) por categoria das 6 features + ruído gaussiano — sem nenhum bônus de interação entre features; os $k = \text{round}(N \times 0,35)$ módulos de maior score recebem SIM. Cada feature é salva em **duas formas**: valor bruto e categoria (baixo/médio/alto).

2 Resultados da validação

2.1 Proporção de classes

Classe	Registros	Proporção
SIM (com defeito)	350.000	35,0%
NÃO (sem defeito)	650.000	65,0%

Equivalente à proporção da versão anterior (34,7%/65,3% com N=150) — o mecanismo top-k (`k = round(N × 0,35)`) garante essa fração independentemente de como as features são geradas. Com N maior, o arredondamento bate exatamente em 35,0%.

2.2 Matriz de correlação de Pearson (features numéricas)

	Complex.	LOC	Autores	Churn	Imports	Cobert.
Complex.	1,000	-0,002	-0,002	-0,000	-0,001	0,000
LOC	-0,002	1,000	-0,000	-0,001	-0,001	-0,001
Autores	-0,002	-0,000	1,000	0,000	-0,000	0,000
Churn	-0,000	-0,001	0,000	1,000	-0,001	0,001
Imports	-0,001	-0,001	-0,000	-0,001	1,000	-0,001
Cobert.	0,000	-0,001	0,000	0,001	-0,001	1,000

A independência ficou ainda mais evidente com N maior

O maior $|r|$ de toda a matriz é **0,002** (complexidade $\times$ LOC) — ruído amostral de uma amostra de 1.000.000 de registros (erro-padrão de correlação nula $\approx 1/\sqrt{(N-2)} \approx 0,001$ nesta escala), não dependência estrutural. Todos os 15 pares têm $|r| \leq 0,002$ — uma ordem de grandeza abaixo do $|r|$ máximo com $N=150$ (0,087, erro-padrão esperado $\approx 0,082$). Aumentar N não muda a correlação populacional (que é 0) — só reduz a variância da estimativa amostral, então o $|r|$ observado converge para 0. Confirma que o gerador implementa de fato a decisão "Naive Bayes puro".

Complexidade $\times$ LOC = -0,002: no dataset original deste projeto (anterior à decisão "Naive Bayes puro") essa correlação era 0,67 (programada de propósito, citando Shepperd, 1988); aqui continua deliberadamente nula, e com $N=1.000.000$ fica visivelmente mais próxima de 0 do que já estava com $N=150$. A discussão sobre a correlação real que a literatura documenta é o assunto central da Reflexão Crítica da Etapa 4.

2.3 Estatísticas descritivas (valor bruto de cada feature)

Feature	Média	Desvio	Mín.	Mediana	Máx.
Complexidade ciclomática	12,72	11,49	1	9,0	120
LOC	140,39	160,19	8	90,0	1500
Nº de autores	2,55	1,24	1	2,0	9
Churn relativo (%)	13,83	11,55	0,4	10,5	85,0
Nº de imports	7,76	5,24	0	6,0	40
Cobertura de testes (%)	62,80	18,50	2,0	63,0	99,0

LOC e complexidade continuam assimétricas à direita (média $>$ mediana), como esperado — isso não mudou; o que mudou é que as duas deixaram de crescer juntas (§2.2).

2.4 Taxa de defeito por categoria, feature a feature

O rótulo ainda depende de cada feature isoladamente — só deixou de haver dependência entre features.
Taxa geral = 35,0%:

Feature	Baixo	Médio	Alto
Complexidade ciclomática	25%	43%	59%
Nº de autores	21%	32%	58%
Churn relativo	25%	41%	63%
Nº de imports	28%	33%	42%
LOC	44%	30%	36%
Cobertura de testes	40%	34%	30%

- **Complexidade, autores, churn e imports:** taxa cresce de forma monótona com a categoria — coerente com os pesos do gerador. Ficaram ligeiramente diferentes das taxas com N=150 (ruído amostral menor), mas a forma monótona é a mesma.
- **LOC:** relação **não monótona, em U** (baixo 44%, médio 30%, alto 36%) — de propósito. Reflete Koru et al. (2008): módulos pequenos são proporcionalmente mais propensos a defeito. Propriedade de LOC *isolado*, não depende de correlação com outra feature.
- **Cobertura de testes:** sinal fraco, sem tendência clara (40%/34%/30%) — reproduz de propósito a fraqueza documentada na literatura.

3 Conformidade com os requisitos da atividade

Requisito	Situação
Pelo menos 100 registros	1.000.000 de registros (<code>N_REGISTROS = 1_000_000</code>)
Padrões intencionais e realistas, não aleatórios	Cada feature segue distribuição deliberada (§1.4); o rótulo depende das 6 de forma coerente com a literatura (§2.4) — o que não é intencional, por decisão, é a correlação <i>entre</i> features (§1.2)
Distribuição razoável entre classes	35,0% / 65,0% (≈ 1:2)
Usar apenas as features discretizadas da Etapa 1	6 features, mesmas faixas; CSV traz valor bruto e categoria
Dados salvos em tabela incluída no entregável	<code>dados/etapa2_dados_treinamento.csv</code> (1.000.000 linhas). Grande, reproduzível via seed fixa — não versionado no Git

Reprodutibilidade

Rodar `python3 dados/gerar_dados.py` regenera exatamente o mesmo CSV (semente fixa `SEED = 42`, ramificada por feature via `rng.spawn()`). Com N=1.000.000 e geração totalmente vetorizada em numpy (sem laço `for` linha a linha), o script roda em poucos segundos.

Nota sobre o uso de IA

O script, os parâmetros do modelo generativo e este relatório foram construídos em diálogo com uma IA generativa. A decisão de tornar as features independentes ("Naive Bayes puro") foi tomada e documentada no CLAUDE.md antes desta geração; os pesos do rótulo foram derivados das referências verificadas na Etapa 1 e **conferidos na saída de validação** antes de fixar os valores. Todo o código é comentado e defensável linha a linha.